MIRAI BOTNET ATTACK TRAFFIC ANALYSIS WITH MACHINE LEARNING

by

Mack Preston

Submitted in partial fulfillment of the
requirements for the degree of
Bachelor of Computer Science, Honours

at

Dalhousie University
Halifax, Nova Scotia
April 2018

© Copyright by Mack Preston, 2018

DALHOUSIE UNIVERSITY

FACULTY OF COMPUTER SCIENCE

The undersigned hereby certify that they have read and recommend to the Faculty of Computer Science for acceptance a thesis entitled "MIRAI BOTNET ATTACK TRAFFIC ANALYSIS WITH MACHINE LEARNING" by Mack Preston in partial fulfillment of the requirements for the degree of Bachelor of Computer Science, Honours.

Dated: April 10, 2018

Supervisor: _____
N. Zincir-Heywood

Reader: _____
M. Heywood

DALHOUSIE UNIVERSITY

DATE: April 10, 2018

AUTHOR: Mack Preston

TITLE: MIRAI BOTNET ATTACK TRAFFIC ANALYSIS WITH MACHINE LEARNING

DEPARTMENT OR SCHOOL: Faculty of Computer Science

DEGREE: B.C.Sc. (Honours) CONVOCATION: May YEAR: 2018

Permission is herewith granted to Dalhousie University to circulate and to have copied for non-commercial purposes, at its discretion, the above title upon the request of individuals or institutions. I understand that my thesis will be electronically available to the public.

The author reserves other publication rights, and neither the thesis nor extensive extracts from it may be printed or otherwise reproduced without the author's written permission.

The author attests that permission has been obtained for the use of any copyrighted material appearing in the thesis (other than brief excerpts requiring only proper acknowledgement in scholarly writing), and that all such use is clearly acknowledged.

Signature of Author

Table of Contents

List of Tables . . . . . **vi**

List of Figures . . . . . **viii**

Abstract . . . . . **ix**

Acknowledgements . . . . . **x**

Chapter 1 Introduction and Background **1**

Chapter 2 Literature Review **3**

2.1 Motivating a Market or Regulatory Solution to IoT Insecurity 3

2.2 Scalable High-Interaction Physical Honeypots 4

2.3 Analysis of Mirai 5

2.4 Literature Review - Summary 5

Chapter 3 Capturing Attack Traffic **7**

3.1 Setting up the Environment 7

3.2 Conscripting New Devices Into the Botnet 12

3.3 Attack Traffic Generation 12

3.4 Flow Exporters 13

3.4.1 Argus 14

3.4.2 Tranalyzer 14

Chapter 4 Attack Detection with Snort IDS **15**

4.1 Snort IDS 15

4.2 Testing IDS Rulesets with Mirai Traffic 15

Chapter 5 Attack Traffic Analysis with Machine Learning **18**

5.1 Datasets 18

5.1.1 6-Attack Dataset 18

5.1.2 CTU Dataset 19

5.2 Preprocessing 20
5.3 Decision Tree Algorithms 21
5.4 Phase 1 - Decision Tree Classification of 6-Attack Datasets 22
5.5 Phase 2 - Decision Tree Classification of Mixed Datasets 26
Chapter 6 Conclusion and Future Work 30
6.1 Conclusion 30
6.2 Future Work 30
Bibliography 33
Appendix A Confusion Matrices 34

List of Tables

Table 3.1 Captured Mirai attack traffic statistics, '-' means no attack traffic was generated 13

Table 5.1 Features from Argus that were used for building decision trees. 19

Table 5.2 Features from Tranalyzer that were used for building decision trees. 19

Table 5.3 Class counts for mixed dataset (CTU and 6-Attack-Argus) . . . 20

Table 5.4 Performance measures for WEKA decision tree classification of 10-fold cross-validated 6-Attack-Argus dataset. 23

Table 5.5 Performance measures for WEKA decision tree classification of 10-fold cross-validated 6-Attack-Tranalyzer dataset. 24

Table 5.6 Performance measures for WEKA decision tree classification of the 6-Attack-Argus dataset. 24

Table 5.7 Performance measures for WEKA decision tree classification of the 6-Attack-Tranalyzer dataset. 25

Table 5.8 Performance measures for WEKA decision tree classification of the balanced mixed dataset. 27

Table 5.9 Performance measures for WEKA decision tree classification of the unbalanced mixed dataset. 28

Table A.1 WEKA 10-fold cross-validated decision tree confusion matrix for 6-Attack-Argus dataset. 34

Table A.2 WEKA decision tree confusion matrix for 6-Attack-Argus dataset. 34

Table A.3 WEKA 10-fold cross-validated decision tree confusion matrix for 6-Attack-Tranalyzer dataset. 35

Table A.4 WEKA decision tree confusion matrix for 6-Attack-Tranalyzer dataset. 35

Table A.5 WEKA 10-fold cross-validated decision tree confusion matrix for unbalanced mixed dataset. 35

Table A.6 WEKA decision tree confusion matrix for unbalanced mixed dataset. 36

Table A.7 WEKA decision tree confusion matrix for balanced mixed dataset. 36

List of Figures

Figure 3.1 VirtualBox environment configuration 8

Figure 3.2 Virtual Network Setup 8

Figure 3.3 C&C Database Schema 10

Figure 3.4 Telnet session into C&C server, showing successful login and attack options 11

Figure 4.1 File sizes of the Snort logs produced when the Mirai traffic was processed using the rule proposed by Sinanovic and Mrdovic. This shows that only syn traffic generated alerts. 16

Figure 4.2 File sizes of the Snort logs produced when the Mirai traffic was processed using the community ruleset. This shows that no alerts were generated for any of the attacks. 16

Figure 4.3 Partial Snort log of alerts produced when syn attack traffic was processed by the rule proposed by Sinanovic and Mrdovic. A total of 204388 alerts were raised in this log. 17

Figure 5.1 Sizes, counts and content of each of the CTU-13 scenarios. Scenario 5, highlighted above, was selected for analysis in this thesis. 20

Figure 5.2 Decision tree generated by 10-fold cross validation on the 6-Attack dataset using Argus features. 22

Figure 5.3 Decision tree generated by 10-fold cross validation on the 6-Attack dataset using Tranalyzer features. 23

Figure 5.4 Decision tree generated by running 66/33 (train/test) split on the 6-Attack dataset using Argus features. 24

Figure 5.5 Decision tree generated by running 66/33 (train/test) split on the 6-Attack dataset using Tranalyzer features. 25

Figure 5.6 Decision tree generated from balanced mixed dataset. 27

Figure 5.7 Decision tree generated from unbalanced mixed dataset. 28

Abstract

Mirai is a malware that infects Internet of Things (IoT) devices, enlisting them into a botnet that can be used to launch Distributed Denial of Service (DDoS) attacks. The malware spreads by scanning for other vulnerable devices and using Telnet and a list of default passwords to gain access. At its peak, Mirai infected over 100,000 devices globally and was used to launch attacks against journalists, cloud hosting providers and popular web services such as Netflix, Twitter and Amazon. The number of IoT devices is growing rapidly, and the inability to recall or update them proves the need for stronger attack mitigation strategies. In this work, an air-gapped, virtual testbed was created, a host was infected with Mirai, and a variety of DDoS attacks were launched and captured. The resulting traffic was then tested against current open-sourced Intrusion Detection System (IDS) solutions. Finally, C4.5 and CART decision trees were used to classify Mirai's DDoS attack traffic. The classifiers were successful in distinguishing the Mirai traffic from normal and other botnet traffic with very low false alarm rates.

Acknowledgements

Thanks to my honours supervisor, Dr. Nur Zincir-Heywood, for her help and guidance throughout the last year. Thanks also to my reader, Dr. Malcolm Heywood.

Chapter 1

Introduction and Background

The Mirai botnet is a malware that exploits vulnerabilities in Internet of Things devices such as surveillance cameras, home routers, DVRs and baby monitors. The affected devices are given commands from a control server, which is operated by the botnet owner. A network of compromised devices can be used for a variety of purposes, including harvesting credentials, spamming and launching DDoS attacks. Botnet owners often rent out their botnets, offering attacks-as-a-service to anyone looking to take down a site.

The Mirai botnet set record attack volumes in 2016, taking down several high-value sites and services. The security blog KrebsOnSecurity was temporarily taken down with a DDoS attack weighing in at an estimated 620 Gbps [1]. French cloud hosting service OVH, and DNS (Domain Name System) host Dyn, were both hit with Mirai attacks upwards of 1 Tbps [2]. The attack on Dyn took some of the most popular sites offline, including Netflix, Twitter and Amazon. An estimated 100,000 Mirai infected IoT devices were used in these attacks. The success of the botnet is mostly attributed to the vast number of insecure IoT devices. The number of IoT devices is expected to exceed 50 Billion by 2020. Many of these are being produced quickly, cheaply and without concern for security. This highlights the growing need for detection and mitigation strategies against IoT botnets.

This work aims to better understand the Mirai botnet and to investigate the applicability of machine learning techniques in identifying Mirai DDoS attack traffic. To effectively test current IDS solutions and machine learning approaches, a virtual environment needed to be setup to capture Mirai traffic. The remainder of the thesis is organized as follows. Chapter 2 summarizes the related work performed in the

[1] KrebsOnSecurity: https://krebsonsecurity.com/2016/10/hacked-cameras-dvrs-powered-todays-massive-internet-outage/

[2] Attack on OVH: https://f5.com/labs/articles/threat-intelligence/ddos/mirai-the-iot-bot-that-took-down-krebs-and-launched-a-tbps-attack-on-ovh-22422

literature on Mirai Botnet. Chapter 3 discusses the analysis of the publicly available Mirai code, and how it is used to generate and capture traffic in a controlled lab environment. Chapter 4 introduces the analysis and detection of the attacks using Snort intrusion detection system on the captured traffic, whereas Chapter 5 presents the evaluation and results of the analysis of the same traffic using decision tree machine learning algorithms. Finally conclusions are drawn and the future works are discussed in Chapter 6.

Chapter 2

Literature Review

This chapter will give brief summaries of the related works.Section 2.1 will discuss work done by Jerkins [6] around the the need for regulatory solutions to IoT security. Section 2.2 will summarize work by Guarnizo et al. [3] concerning a better method of capturing botnet traffic with honeypots . Finally, Section 2.3 will discuss work by Sinanovic and Mrdovic [9]. Their work involved using Mirai to launch attacks in a virtual environment and tailoring custom Snort rules to detect the resulting traffic.

2.1 Motivating a Market or Regulatory Solution to IoT Insecurity

Botnets used to be controlled from a single Command and Control (C&C) server, but detection techniques forced modern botnets into using HTTP and using a peer-to-peer control structure. Two main defense techniques to defend against DDoS are shutting down the C&C server, and sinkholing the traffic. Shutting down the C&C server can be difficult because the communication channels of botnets are often encrypted, making the C&C servers hard to find. They often employ tactics such as fast-flux to further disguise their networks. These tactics paired with the jurisdictional limitations can increase the time it takes to find and take down any C&C servers. The second defense method, sinkholing, involves identifying malicious traffic and filtering it at various stages upstream [6].

Two key stakeholders are currently enabling the mass production of insecure IoT devices. The buyers often don't know about the insecurity of these devices. Most buyers are primarily concerned with the cost and ease of use. Since these devices are being used to attack someone else and not the owner, the owner is less motivated to act. Since many customers are not actively looking for this additional security, there is little incentive for companies to spend extra time and resources developing it. Many customers also don't want to go through administrative setup and password resets. This makes secure devices potentially less attractive than their cheaper, easier

to use alternatives.

An interesting feature of Mirai is how it deals with other botnets. Once Mirai has infected a device, it shuts down all vulnerable access points, shutting down SSH, Telnet and HTTP. This protects the device from being taken over by another botnet (including another Mirai botnet controlled by a different botmaster). It then begins to scan the network looking for new devices to infect. A device can be disinfected by turning it off but is quickly re-infected by the other infected devices on the network.

In [6], Jerkins modified a version of the botnet, using it to catalog infected devices and motivate their owners, manufacturers and law enforcement to re-secure the devices. This version uses Mirai's territorial nature to protect the vulnerable interfaces from other botnets. All attack related functionality was removed but the infection mechanisms were left intact. In this case, the C&C server is only used to store the information of the compromised devices for law enforcement and manufacturers. The authors note that this approach currently lacks the legal framework required to allow unauthorized access to public devices across multiple jurisdictions.

2.2 Scalable High-Interaction Physical Honeypots

Honeypots are generally used to gain information about security threats without putting critical infrastructure at risk. A honeypot needs to offer realistic interaction to potential attackers, leading the attacker to believe it is a legitimate server [3].

Low interaction honeypots involve offering partial TCP and IP services to the attacker without giving them access to an actual system. These are easy to detect as artificial due to their simplicity. Furthermore, they do not offer much insight from the actions of an attacker. High interaction honeypots involve offering actual devices or detailed emulations to the attacker, where the attacker can interact with the device's full functionality. High interaction honeypots can be expensive and difficult to maintain and scale, especially when attempting to accurately emulate a wide range of IoT devices.

Guarnizo et al. propose the implementation of high interaction honeypots as geographically distributed, physical devices [3]. These devices would use non-malicious wormholes to relay interaction data and threat intelligence back to the honeypot operator. Instead of placing physical devices across the globe, the locations would be

spoofed via wormholes connecting the devices to edge nodes hosted by cloud providers.

They discuss Shodan, a search engine that lists IoT devices that it finds on the Internet. The service uses unknown heuristics to determine if a found device is a honeypot or not. While many honeypots are marked as (likely a honeypot), on average, the physical honeypots of Guarnizo et al. received low (not likely a honeypot) scores [3].

2.3 Analysis of Mirai

After the release of Mirai's source code, static analysis became easier. The botnet consists of three components, the bot (written in C), the C&C server (written in Golang) and the loader (written in C) [9]. An infected device will wait for attack commands from the C&C server. Once it gets a command to initiate an attack, it will spam the target address with packets of the desired attack type. The bot is also responsible for infecting new devices. When a device is infected, it runs a 'killer' module, which kills other botnets and older versions of Mirai. It also shuts down applications running on ports 22, 23 and 80, preventing future infection by other botnets. To infect new devices, the bot uses a scanner to locate vulnerable devices. It randomly selects IP addresses, checks to make sure they are not on Mirai's white-list, then attempts to establish a Telnet connection. If the connection is established, Mirai loops through its list of factory default credentials and tries to authenticate itself. Sinanovic and Mrdovic go on to generally explain the functionality of the loader and C&C server (which will be explained in more detail later on) [9].

Sinanovic and Mrdovic also perform a dynamic analysis. For this purpose, a virtual environment was set up using VirtualBox. TCP 'syn flood' attack traffic was generated and analyzed. This allowed for the creation Snort IDS rules to raise alerts for the traffic.

2.4 Literature Review - Summary

Jerkins [6] showed the massive scope of the issues surrounding IoT botnets, as well as the difficulty regulating against poor security practices, recalling products and shutting down C&C servers. This showed the need to explore alternate avenues,

particularly the mitigation of attacks from infected devices. Guarnizo et al. [3] showed the applicability of high-interaction honeypots for studying this problem. Sinanovic and Mrdovic [9] performed high level static anlaysis, as well as some dynamic analysis of the malware in a virtual environment. They also derived a rule from the analysis that could be used in Snort IDS to identify the attack traffic.

A solution like Guarnizo's would not be feasible in the time frame of this project. Furthermore, traffic gathered in honeypots is very difficult to label confidently. The work done by Sinanovic and Mrdovic presents an opportunity to expand, elaborate and confirm. Rules were created for a single attack but could be created for each of the 10. Furthermore, no results were presented for the effectiveness of the rules, especially if the traffic contained legitimate behavior as well as other attack behaviors. It also does not account for the numerous active and future variants of Mirai. In generating new attack traffic data in a new virtual environment, the static and dynamic analysis could be confirmed.

Chapter 3

Capturing Attack Traffic

To perform analysis on Mirai, attack traffic first needed to be captured. To the best of my knowledge, at the time of writing, there is no public dataset with labelled Mirai traffic. Since a variant of Mirai's source code was leaked, the code could be compiled and run, and genuine, labelable attack traffic could be generated [3]. This chapter discusses how the attack traffic was successfully generated. Section 3.1 lays out how the virtual environment and network was setup, as well as setting up the required libraries, frameworks and virtual infrastructure necessary to capture the Mirai attack traffic. Section 3.2 discusses how Mirai scans for and infects other devices, and Section 3.3 shows how DDoS attacks were launched and captured. Finally, Section 3.4 explains the use of flow exporters in converting raw traffic captures to network flows.

3.1 Setting up the Environment

Oracle's VirtualBox was used to simulate a small network, consisting of virtual machines for a router, a C&C server and two clients. VirtualBox's internal network feature was used to connect the virtual machines. Only one internal network was established, linking all virtual machines on a single bus. This made initial setup easier, allowing all devices to communicate directly with each other, rather than configuring separate interfaces, and internal networks for each connection to the router.

To ensure the infection did not spread outside the lab environment, the computer hosting the virtual machines was taken off the network. All required software on the host machine as well as the virtual machines were installed, setup and tested before taking the computer off the network. VirtualBox's cloning functionality was used to make copies of the client machine, already preloaded with the required software and configurations. This allowed for scaling the test network without an Internet connection.

[3] Mirai Source Code Leak: https://github.com/jgamblin/Mirai-Source-Code

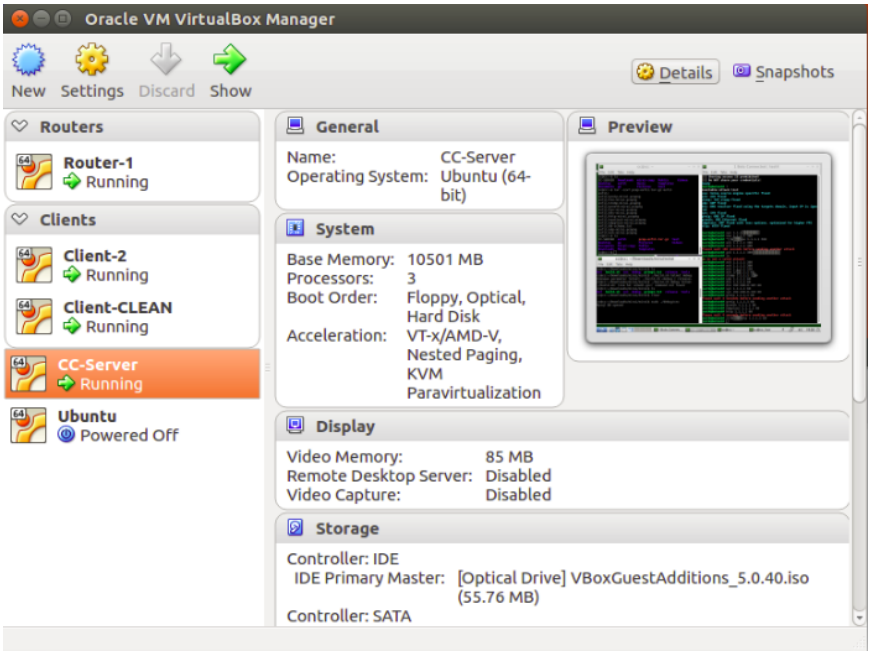

Figure 3.1: VirtualBox environment configuration

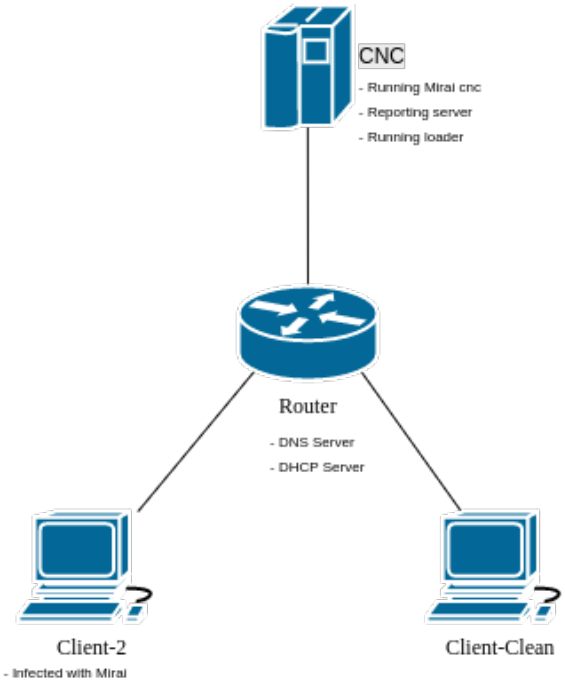

Figure 3.2: Virtual Network Setup

All virtual machines were loaded with the Lubuntu operating system and connected to the internal network 'intnet-1' on their 'enp0s8' interface [4]. Two clients were setup to emulate IoT devices, using a lightweight version of Ubuntu, called Lubuntu for an operating system. One of these devices, 'Client-2', was manually infected by running the i386 compiled bot code. The other device, 'Client-Clean', was intended to be used as a clean IoT device that would be infected by 'Client-2'. The Virtualbox configuration can be seen in Figure 3.1, and the network configuration can be seen in Figure 3.2. The router was set up with a DNS server, and a DHCP server. The DNS server was necessary because the bots are hard-coded to contact the control server based on its DNS entry, instead of a raw IP address. This allows botnet operators to move the C&C server more easily, making it more resilient to take-down attempts. A DNS host record for the C&C server was configured manually. The C&C host name from the host record was also encrypted using Mirai's 'enc' utility and hard-coded into the bot code. DHCP was implemented to allow for easy scaling of the testbed network size, but scaling did not end up being necessary for the experiments in this thesis [5]. Mirai's 'C&C' and 'loader' binaries were run on the C&C machine. When a bot's scanner successfully logs into another vulnerable IoT device, it contacts the reporting server (the server running loader) and sends it the new vulnerable device's information. For this work, the reporting server and C&C server were setup on the same machine for simplicity. Once the reporting server obtains details about a new vulnerable device, it connects over Telnet and delivers the bot payload over WGET or TFTP. The C&C server was setup by installing Go, MySQL and other dependencies [6].

The MySQL server on the C&C server stores user information on patrons of the botnet-as-a-service. The C&C server listens for Telnet on port 23, providing users a login prompt, and a command interface for launching attacks upon successful login. Users were manually inserted into the database, so that attacks could be launched. The database stores users, attack history and white-listed hosts. The C&C

[4] Lubuntu Homepage: https://lubuntu.net/downloads/

[5] Useful reference for configuring network interfaces, DNS and DHCP: https://killtacknine.com/building-an-ubuntu-16-04-router-part-1-network-interfaces/

[6] Useful setup guide for installing C&C and bot dependencies, cross-compiling binaries, setting up the C&C database and editing the encrypted addresses hardcoded into Mirai: https://www.cdxy.me/?p=746

```
3 rows in set (0.00 sec)

mysql> describe users;
+----------------+------------------+------+-----+---------+----------------+
| Field          | Type             | Null | Key | Default | Extra          |
+----------------+------------------+------+-----+---------+----------------+
| id             | int(10) unsigned | NO   | PRI | NULL    | auto_increment |
| username       | varchar(32)      | NO   | MUL | NULL    |                |
| password       | varchar(32)      | NO   |     | NULL    |                |
| duration_limit | int(10) unsigned | YES  |     | NULL    |                |
| cooldown       | int(10) unsigned | NO   |     | NULL    |                |
| wrc            | int(10) unsigned | YES  |     | NULL    |                |
| last_paid      | int(10) unsigned | NO   |     | NULL    |                |
| admin          | int(10) unsigned | YES  |     | 0       |                |
| intvl          | int(10) unsigned | YES  |     | 30      |                |
| api_key        | text             | YES  |     | NULL    |                |
+----------------+------------------+------+-----+---------+----------------+
11 rows in set (0.00 sec)

mysql> describe history;
+-----------+------------------+------+-----+---------+----------------+
| Field     | Type             | Null | Key | Default | Extra          |
+-----------+------------------+------+-----+---------+----------------+
| id        | int(10) unsigned | NO   | PRI | NULL    | auto_increment |
| user_id   | int(10) unsigned | NO   | MUL | NULL    |                |
| time_sent | int(10) unsigned | NO   |     | NULL    |                |
| duration  | int(10) unsigned | NO   |     | NULL    |                |
| command   | text             | NO   |     | NULL    |                |
| max_bots  | int(11)          | YES  |     | -1      |                |
+-----------+------------------+------+-----+---------+----------------+
6 rows in set (0.00 sec)

mysql> describe whitelist;
+---------+---------------------+------+-----+---------+----------------+
| Field   | Type                | Null | Key | Default | Extra          |
+---------+---------------------+------+-----+---------+----------------+
| id      | int(10) unsigned    | NO   | PRI | NULL    | auto_increment |
| prefix  | varchar(16)         | YES  | MUL | NULL    |                |
| netmask | tinyint(3) unsigned | YES  |     | NULL    |                |
+---------+---------------------+------+-----+---------+----------------+
```

Figure 3.3: C&C Database Schema

database schema can be found in Figure 3.3. The botnet also comes with build scripts, allowing the botnet to be compiled in either debug or release mode. Release mode applies obfuscation techniques to avoid detection such as scrambling symbol tables and preventing debugger attachment. Debug mode provided more verbose program output but prevented the bot from launching attacks or scanning for other infected devices. An altered debug mode was used for testing, allowing for easier debugging. The lines preventing scanning and attacks in debug mode were commented out. Both modes support cross-compiling for various architectures, but this research was limited to x86 [7].

[7]Useful setup guide for installing C&C and bot dependencies, cross-compiling binaries, setting up the C&C database and editing the encrypted addresses hardcoded into Mirai https://www.cdxy.me/?p=746

```
я люблю куриные наггетсы
пользователь: test4
пароль: *****

проверив счета...
[+] DDOS | Succesfully hijacked connection
[+] DDOS | Masking connection from utmp+wtmp...
[+] DDOS | Hiding from netstat...
[+] DDOS | Removing all traces of LD_PRELOAD...
[+] DDOS | Wiping env libc.poison.so.1
[+] DDOS | Wiping env libc.poison.so.2
[+] DDOS | Wiping env libc.poison.so.3
[+] DDOS | Wiping env libc.poison.so.4
[+] DDOS | Setting up virtual terminal...
[!] Sharing access IS prohibited!
[!] Do NOT share your credentials!
Ready
test4@botnet# ?
Available attack list
vse: Valve source engine specific flood
ack: ACK flood
stomp: TCP stomp flood
udp: UDP flood
dns: DNS resolver flood using the targets domain, input IP is igno
red
syn: SYN flood
greip: GRE IP flood
greeth: GRE Ethernet flood
udpplain: UDP flood with less options. optimized for higher PPS
http: HTTP flood

test4@botnet#
```

Figure 3.4: Telnet session into C&C server, showing successful login and attack options

3.2 Conscripting New Devices Into the Botnet

The connection between an infected device and the C&C server was captured with Wireshark [8]. When a bot is first infected, it reaches out and registers itself with the C&C over Telnet. The C&C then performs intermittent health-checks on the bot via Telnet to see if the bot is still operational. A Telnet server was installed on 'Client-Clean', which had credentials 'client:admin'. These credentials were added to Mirai's default credential list and the other 62 credentials were commented out. The random IP generator for deciding which IP address to try and infect next was hard-coded to always return the IP address of 'Client-Clean'. This was done to reduce the time it took for the scanner to find Client-Clean, and to reduce randomness in testing.

When analyzing the traffic with Wireshark, the infected device can be seen establishing a Telnet connection with 'Client-Clean', but the Telnet session is immediately closed. This is possibly due to the quirks of VirtualBox networking and the network configuration discussed earlier. This issue may also be caused by incompatibilities between Mirai's custom Telnet implementation and Lubuntu's Telnet clients and servers. Since this project was time sensitive, the scope shifted solely to DDoS attack traffic generation and analysis. Other than this issue, results for dynamic and static analysis were consistent with the findings of other authors. [9]

3.3 Attack Traffic Generation

Mirai can launch several types of DDoS attacks. Dumpcap [9] was used to capture traffic on the infected device, 'Client-2'. This work focused on capturing traffic from a single infected host, rather than a full botnet. We are not trying to classify whether a DDoS is happening, but instead whether a flow from a single host is Mirai traffic. For this reason, only outbound traffic of the infected bot was needed. To capture attack traffic, an attack first needed to be initiated by a user. Using the credentials 'test4:test4' (inserted into the database earlier), a Telnet session to the C&C server was created, resulting in the attack prompt. The Mirai command prompt can be seen in Figure 3.4. Entering '?' shows available attacks. Entering the attack type, target IP address and duration in seconds initiates the attack. It is worth noting that

[8] Wireshark Homepage: https://www.wireshark.org/
[9] Dumpcap Homepage: https://www.wireshark.org/docs/man-pages/dumpcap.html

entering an invalid attack seems to increase the user's cooldown time (the time they have to wait between launching attacks). For this reason, the 'adduser' command was useful to create a new admin user account, rather than connecting using MySQL insert or update statements manually.

Attack traffic was captured for each of the 10 attacks, some of which did not produce any traffic. This could be because of network configuration, or functionality blocked in debug mode. Either way, more static analysis is required to fix this issue. The other attacks, ('ack', 'syn', 'vse', 'udp', 'greip' and 'greeth'), produced a massive amount of attack traffic. The duration of the attacks ranged from 10 to 300 seconds. For the analysis in Chapter 5, these captures were all trimmed down to 10 seconds, starting when the Telnet attack command was issued. Attack traffic captures were then converted to flows. All attack traffic capture sizes, durations, packet and flow counts can be seen in Table 3.1.

Table 3.1: Captured Mirai attack traffic statistics, '-' means no attack traffic was generated

Attack	pcap File Size (MB)	Trimmed Duration (s)	Number of Packets	Number of Flows (Argus)	Number of Flows (Tranalyzer)
vse	23	10	233971	63666	63664
ack	111	10	193794	193786	193786
udp	96	10	170420	170408	170407
syn	20	10	193049	193041	193040
greip	116	10	198400	198394	198394
greeth	136	10	227533	227526	227524
udpplain	-	-	-	-	-
http	-	-	-	-	-
dns	-	-	-	-	-
stomp	-	-	-	-	-

3.4 Flow Exporters

Flow exporters take raw traffic and aggregate the packets into flows. A flow is roughly equivalent to a call or connection and can be identified using properties from the packet header. Flows reduce the complexity of the traffic while preserving the meaningful information about the communication. Additionally, flow exporters calculate

statistics about traffic in a flow, providing more descriptive features that can be used to analyze the traffic [4]. This is especially useful when employing machine learning techniques because patterns can emerge from the new calculated features. For this paper, both Argus and Tranalyzer were used as flow exporters, resulting in two datasets with different feature sets being generated from the same traffic. As can be seen in Table 3.1, both flow exporters seem to have a roughly 1-to-1 mapping between packet and generated flow, with the exception of the 'vse' protocol.

3.4.1 Argus

Argus is a real-time flow monitor that can perform network traffic auditing [10]. For this paper, Argus 3.0 was used as a flow exporter to extract features from pcap files. The features extracted from the traffic by Argus are described in Section 5.2.

3.4.2 Tranalyzer

Tranalyzer is another flow generator and extends NetFlow's functionality [11]. It is newer than Argus, has a larger set of features and calculated statistics but seems more complicated. For this thesis, Tranalyzer2 was used as a flow exporter to extract features from pcap files [12]. The features extracted from the traffic by Tranalyzer are described in Section 5.2.

[10] Argus Download and Documentation: https://qosient.com/argus/index.shtml
[11] Tranalyzer Download and Documentation: https://tranalyzer.com
[12] Exact Argus and Tranalyzer commands used in this thesis can be found in the 'run.sh' file at https://github.com/MackPreston/mirai-thesis

Chapter 4

Attack Detection with Snort IDS

Intrusion Detection Systems (IDS) are often deployed to aid in the mitigation of attacks like DDoS, using (usually hand-crafted) signatures to identify malicious traffic. The difference between an IDS and an Intrusion Prevention System (IPS) is a subtle one. While an IDS focusses primarily on identification and alerting and reporting of malicious traffic, an IPS can take actions such as actively blocking or dropping the traffic. This thesis will focus on IDS and detection, knowing that it could be extended later if the false-positive rate were low enough. Section 4.1 will introduce Snort, the IDS used for testing the Mirai traffic against current mitigation solutions. Section 4.2 will present the results of my experiments with Snort on Mirai traffic.

4.1 Snort IDS

Snort is an open source IDS that detects malicious traffic based on known signatures. It works by analyzing network traffic, either online or offline. This means that Snort can be used to analyze a capture file of traffic that has already occurred, or it can be bound to an interface and process live traffic as it comes in. Snort generates alerts by applying user defined rules to traffic. For this thesis, Snort 3.0 was used, and the Mirai attack traffic was fed in as pcap files [8].

4.2 Testing IDS Rulesets with Mirai Traffic

Two Snort rulesets were used for testing the raw Mirai traffic captured in Chapter 2. The first ruleset is 'snort3-community-rules', which is a free, publicly available ruleset with a broad range of rules [13]. The second ruleset contains a single rule proposed by Sinanovic and Mrdovic [9]. This rule is meant as a proof of concept, showing that rules could be handcrafted based on manual expert inspection of Mirai's attack

[13] Snort3 Community Rules: https://www.snort.org/downloads/#rule-downloads

```
mack@mack-K501UX:~/workspace/mirai/ids-logs/bosnia$ ls -lh
total 21M
-rw-rw-r-- 1 mack mack   0 Jan 17 20:00 ack-alerts.csv
-rw-rw-r-- 1 mack mack   0 Jan 17 20:00 greeth-alerts.csv
-rw-rw-r-- 1 mack mack   0 Jan 17 20:01 greip-alerts.csv
-rw-rw-r-- 1 mack mack 21M Jan 17 20:01 syn-alerts.csv
-rw-rw-r-- 1 mack mack   0 Jan 17 20:01 udp-alerts.csv
-rw-rw-r-- 1 mack mack   0 Jan 17 20:01 vse-alerts.csv
```

Figure 4.1: File sizes of the Snort logs produced when the Mirai traffic was processed using the rule proposed by Sinanovic and Mrdovic. This shows that only syn traffic generated alerts.

```
mack@mack-K501UX:~/workspace/mirai/ids-logs/snort3-community$ ls -lh
total 0
-rw-rw-r-- 1 mack mack 0 Jan 17 20:02 ack-alerts.csv
-rw-rw-r-- 1 mack mack 0 Jan 17 20:02 greeth-alerts.csv
-rw-rw-r-- 1 mack mack 0 Jan 17 20:03 greip-alerts.csv
-rw-rw-r-- 1 mack mack 0 Jan 17 20:03 syn-alerts.csv
-rw-rw-r-- 1 mack mack 0 Jan 17 20:03 udp-alerts.csv
-rw-rw-r-- 1 mack mack 0 Jan 17 20:03 vse-alerts.csv
```

Figure 4.2: File sizes of the Snort logs produced when the Mirai traffic was processed using the community ruleset. This shows that no alerts were generated for any of the attacks.

packets. The rule is designed to raise alerts if Mirai's 'syn' attack traffic is detected, but similar rules could be written for the other attacks.

When each of the 6 attacks was processed by Snort, the community ruleset showed no alerts. The single rule proposed by Sinanovic and Mrdovic worked as expected, generating 204388 alerts for the 'syn' attack, but none for the other attacks. Since including the empty log files seems unnecessary, these results can be represented by the file sizes of the logs. The file sizes can be seen in Figures 4.1 and 4.2. Partial logs from the IDS can be seen in Figure 4.3. The lack of alerts raised by the community ruleset indicates the need for better rulesets. Section 6.2 will discuss how both rulesets could be further tested [14].

[14]The exact Snort commands used to test the traffic from each attack can again be found at https://github.com/MackPreston/mirai-thesis

```
1 18/01/13-16:44:58.605045, 73, TCP, raw, 60, C2S, 192.168.0.108:9797, 1.1.1.1:28112, 1:10001:1, allow
2 18/01/13-16:44:58.605148, 74, TCP, raw, 60, C2S, 192.168.0.108:52219, 1.1.1.1:30279, 1:10001:1, allow
3 18/01/13-16:44:58.605874, 75, TCP, raw, 60, C2S, 192.168.0.108:22599, 1.1.1.1:51867, 1:10001:1, allow
4 18/01/13-16:44:58.605892, 76, TCP, raw, 60, C2S, 192.168.0.108:51316, 1.1.1.1:28771, 1:10001:1, allow
5 18/01/13-16:44:58.605903, 77, TCP, raw, 60, C2S, 192.168.0.108:11639, 1.1.1.1:59254, 1:10001:1, allow
6 18/01/13-16:44:58.605911, 78, TCP, raw, 60, C2S, 192.168.0.108:17097, 1.1.1.1:39444, 1:10001:1, allow
7 18/01/13-16:44:58.605922, 79, TCP, raw, 60, C2S, 192.168.0.108:42054, 1.1.1.1:24163, 1:10001:1, allow
8 18/01/13-16:44:58.605930, 80, TCP, raw, 60, C2S, 192.168.0.108:30102, 1.1.1.1:11802, 1:10001:1, allow
9 18/01/13-16:44:58.605940, 81, TCP, raw, 60, C2S, 192.168.0.108:57440, 1.1.1.1:1943, 1:10001:1, allow
10 18/01/13-16:44:58.606063, 82, TCP, raw, 60, C2S, 192.168.0.108:16005, 1.1.1.1:23020, 1:10001:1, allow
11 18/01/13-16:44:58.606180, 83, TCP, raw, 60, C2S, 192.168.0.108:39681, 1.1.1.1:61057, 1:10001:1, allow
12 18/01/13-16:44:58.606352, 84, TCP, raw, 60, C2S, 192.168.0.108:4264, 1.1.1.1:50268, 1:10001:1, allow
13 18/01/13-16:44:58.606358, 85, TCP, raw, 60, C2S, 192.168.0.108:55784, 1.1.1.1:37584, 1:10001:1, allow
14 18/01/13-16:44:58.606504, 86, TCP, raw, 60, C2S, 192.168.0.108:37889, 1.1.1.1:6011, 1:10001:1, allow
15 18/01/13-16:44:58.606676, 87, TCP, raw, 60, C2S, 192.168.0.108:3099, 1.1.1.1:48936, 1:10001:1, allow
16 18/01/13-16:44:58.606688, 88, TCP, raw, 60, C2S, 192.168.0.108:55317, 1.1.1.1:65055, 1:10001:1, allow
17 18/01/13-16:44:58.606842, 89, TCP, raw, 60, C2S, 192.168.0.108:28528, 1.1.1.1:47885, 1:10001:1, allow
18 18/01/13-16:44:58.606858, 90, TCP, raw, 60, C2S, 192.168.0.108:30173, 1.1.1.1:33794, 1:10001:1, allow
19 18/01/13-16:44:58.606873, 91, TCP, raw, 60, C2S, 192.168.0.108:35958, 1.1.1.1:61116, 1:10001:1, allow
20 18/01/13-16:44:58.607022, 92, TCP, raw, 60, C2S, 192.168.0.108:2881, 1.1.1.1:46062, 1:10001:1, allow
21 18/01/13-16:44:58.607164, 93, TCP, raw, 60, C2S, 192.168.0.108:56612, 1.1.1.1:12213, 1:10001:1, allow
22 18/01/13-16:44:58.607169, 94, TCP, raw, 60, C2S, 192.168.0.108:41400, 1.1.1.1:34043, 1:10001:1, allow
23 18/01/13-16:44:58.607249, 95, TCP, raw, 60, C2S, 192.168.0.108:57402, 1.1.1.1:25741, 1:10001:1, allow
24 18/01/13-16:44:58.607310, 96, TCP, raw, 60, C2S, 192.168.0.108:56279, 1.1.1.1:4395, 1:10001:1, allow
25 18/01/13-16:44:58.607405, 97, TCP, raw, 60, C2S, 192.168.0.108:11013, 1.1.1.1:62021, 1:10001:1, allow
26 18/01/13-16:44:58.607467, 98, TCP, raw, 60, C2S, 192.168.0.108:19057, 1.1.1.1:12874, 1:10001:1, allow
```

Figure 4.3: Partial Snort log of alerts produced when syn attack traffic was processed by the rule proposed by Sinanovic and Mrdovic. A total of 204388 alerts were raised in this log.

Chapter 5

Attack Traffic Analysis with Machine Learning

This chapter will discuss analysis performed on the traffic captured in Chapter 3. Section 5.1 will discuss the datasets used for the analysis and Section 5.2 will discuss the preprocessing steps taken to prepare the data for training. Next, Section 5.3 will give a brief explanation of the learning algorithms used. The analysis consisted of two phases. Phase-1 involved training decision tree models to classify attack traffic gathered in Chapter 3 as one of the six DDoS attack types. Phase-2 involved performing the same analysis on a mixed dataset. The mixed dataset contains the 6-Attack dataset and another dataset containing normal and other non-Mirai botnet traffic. The goal of phase-2 was to build classifiers that could distinguish between each of the 6 Mirai attacks, normal traffic and the other attack traffic. Sections 5.4 and 5.5 will go over the results of phase-1 and phase-2 respectively.

5.1 Datasets

This Section will outline the two datasets used in this thesis. Section 5.1.1 will discuss the 6-Attack dataset, which was collected in Chapter 2. Section 5.1.2 will discuss the CTU dataset, which was taken from the CTU-13 data archive.

5.1.1 6-Attack Dataset

The 6-Attack dataset consists of flows exported from the six types of DDoS attacks captured in Chapter 3. Flows were exported using both Argus and Tranalyzer, resulting in two variants of the 6-Attack dataset. The features used in the Argus and Tranalyzer variants are shown in Tables 5.1 and 5.2 respectively. Note that these feature lists contain only the features used in the models and do not contain the features that were dropped in preprocessing.

Table 5.1: Features from Argus that were used for building decision trees.

Label	Dur	Proto	RunTime	Mean
SrcPkts	DstPkts	sTtl	TotAppByte	TotBytes
SrcLoad	DstLoad	Loss	SrcLoss	DstLoss
Sum	Min	Max	sTos	TotPkts
SrcBytes	SAppBytes	DstBytes	DAppBytes	Load
Rate	sMeanPktSz	dMeanPktSz	sMaxPktSz	sMinPktSz

Table 5.2: Features from Tranalyzer that were used for building decision trees.

Label	flowInd	duration	numHdrDesc
numHdrs	ethVlanID	l4Proto	macPairs
dstPortClassN	numPktsSnt	numPktsRcvd	numBytesSnt
numBytesRcvd	minPktSz	maxPktSz	avePktSize
stdPktSize	pktps	bytps	pktAsm
bytAsm	ipMindIPID	ipMaxdIPID	ipMinTTL
ipMaxTTL	ipTTLChg	ipOptCnt	tcpPSeqCnt
tcpSeqSntBytes	tcpSeqFaultCnt	tcpPAckCnt	tcpFlwLssAckRcvdBytes
tcpAckFaultCnt	tcpInitWinSz	tcpAveWinSz	tcpMinWinSz
tcpMaxWinSz	tcpWinSzDwnCnt	tcpWinSzUpCnt	tcpWinSzChgDirCnt
tcpOptPktCnt	tcpOptCnt	tcpMSS	tcpWS
tcpTmS	tcpTmER	tcpEcI	tcpBtm
tcpSSASAATrip	tcpRTTAckTripMin	tcpRTTAckTripMax	tcpRTTAckTripAve
tcpRTTAckTripJitAve	tcpRTTSseqAA	tcpRTTAckJitAve	icmpTCcnt
icmpEchoSuccRatio	icmpPFindex	connSip	connDip
connSipDip	connSipDprt	connF	

5.1.2 CTU Dataset

The CTU dataset was taken from Scenario 5 of the CTU-13 data archive that was collected at CTU University, Czech Republic in 2011. The dataset consists of hand-labelled Argus flows. These flows are labelled as 77 variations of 'botnet', 'background' and 'normal'. Specifically, the bi-directional, labelled flows were used for the analysis. Information on the different CTU scenarios can be seen in Figure 5.1, with the chosen scenario highlighted. This dataset uses the same feature set as 6-Attack-Argus, which can be seen in Table 5.1.

Id	Duration(hrs)	# Packets	#NetFlows	Size	Bot	#Bots
1	6.15	71,971,482	2,824,637	52GB	Neris	1
2	4.21	71,851,300	1,808,123	60GB	Neris	1
3	66.85	167,730,395	4,710,639	121GB	Rbot	1
4	4.21	62,089,135	1,121,077	53GB	Rbot	1
5	11.63	4,481,167	129,833	37.6GB	Virut	1
6	2.18	38,764,357	558,920	30GB	Menti	1
7	0.38	7,467,139	114,078	5.8GB	Sogou	1
8	19.5	155,207,799	2,954,231	123GB	Murlo	1
9	5.18	115,415,321	2,753,885	94GB	Neris	10
10	4.75	90,389,782	1,309,792	73GB	Rbot	10
11	0.26	6,337,202	107,252	5.2GB	Rbot	3
12	1.21	13,212,268	325,472	8.3GB	NSIS.ay	3
13	16.36	50,888,256	1,925,150	34GB	Virut	1

Figure 5.1: Sizes, counts and content of each of the CTU-13 scenarios. Scenario 5, highlighted above, was selected for analysis in this thesis.

Table 5.3: Class counts for mixed dataset (CTU and 6-Attack-Argus)

Label	Number of Flows
greeth-argus	227524
greip-argus	198393
ack-argus	193785
syn-argus	193039
udp-argus	170406
vse-argus	63663
normal-ctu	4676
botnet-ctu	901

5.2 Preprocessing

Once the traffic flows were in csv format, all preprocessing was performed with Python and the Pandas library [15]. The 6-Attack dataset was loaded into a Pandas dataframe by reading in the csv of each of the six attacks, labelling it as such, and aggregating it with the others. This was done separately for the Tranalyzer and Argus variants of the 6-Attack dataset. The CTU dataset was simply loaded into a dataframe since it was already labelled and in a single file. Each dataset was then shuffled (within itself) to ensure unbiased training.

Next, the datasets were cleaned of null or empty values. Some of the features of

[15]Pandas Library Homepage: https://pandas.pydata.org/

the 6-Attack dataset were dropped because they were null for every data point. The null values are most likely caused because each flow contained only one packet. The features dropped in the Argus 6-Attack dataset were also dropped from the CTU dataset so that the two could be easily merged later. A few data points were also dropped from 6-Attack Argus due to null values. Since only four data points (records) were dropped, this should not have a significant effect on the results.

Two sets of features were used for training. 6-Attack-Argus and CTU used a subset of features offered by Argus, while 6-Attack-Tranalyzer used a subset of features offered by Tranalyzer. All features with a non-numeric type were dropped because required further study for mapping them into numerical values. This is left for future work. The port numbers, MAC addresses and IP addresses for both source and destination were also dropped, because they could be easily spoofed [4]. The remaining features were used in the classification, and can be seen in Tables 5.1 and 5.2.

5.3 Decision Tree Algorithms

Decision tree is the machine learning algorithm of choice for this thesis because its output is human readable and it has been shown to be successful in other botnet traffic analysis research [4]. The models also take the form of rules that can be easily ported to rulesets for IDS. Two decision tree algorithms were used in this analysis. The first was WEKA's J-48 algorithm, which is a direct clone of the C4.5 decision tree algorithm [7]. The other was a Python implementation of the CART decision tree algorithm, found in the Sklearn package [16].

The C4.5 Decision Tree algorithm is an extension of the ID3 algorithm. Using a top-down, recursive, divide-and-conquer approach, the algorithm partitions the data into smaller and smaller sets based on the information gain ratio. This results in a tree of rules that can be used for classification [5]. The CART algorithm functions similarly but is only able to form binary trees. It also uses Gini impurity instead of gain ratio for making splitting decisions.

[16] sklearn (scikit-learn) Homepage: http://scikit-learn.org/stable/

```
sMeanPktSz <= 554
|   sMeanPktSz <= 80
|   |   Proto <= 6: syn-argus (193043.0/4.0)
|   |   Proto > 6: vse-argus (63662.0)
|   sMeanPktSz > 80: udp-argus (170405.0)
sMeanPktSz > 554
|   sMeanPktSz <= 578
|   |   Proto <= 6: ack-argus (193784.0)
|   |   Proto > 6: greip-argus (198392.0)
|   sMeanPktSz > 578: greeth-argus (227524.0)
```

Figure 5.2: Decision tree generated by 10-fold cross validation on the 6-Attack dataset using Argus features.

5.4 Phase 1 - Decision Tree Classification of 6-Attack Datasets

All experiments in phases 1 and 2 were carried out in both Python and WEKA (CART and C4.5). However, only the WEKA results are shown in this thesis because they have a cleaner formatting and both algorithms produced nearly identical trees with nearly indistinguishable performance. The confusion matrices for all experiments can be found in Appendix A.

The same procedure was used to build models for the Tranalyzer and Argus variants of the 6-Attack dataset. The first model was built using 10-fold cross-validation. This is where the data is split into 10 folds, where one fold is kept out while the other nine are used for training. This is repeated nine more times, with each fold getting a turn sitting out of training. At the end of the 10 trainings, the results are averaged to get a cross-validated accuracy. The tree representations of the cross-validated models for 6-Attack-Argus and 6-Attack-Tranalyzer can be seen in Figures 5.2 and 5.3 respectively. Their accuracies can be found in Tables 5.4 and 5.5 respectively.

After the datasets were validated, models were trained using a 66/33 training/testing split. 66% of the dataset was used to train the model, and 33% was used to test the model. The tree representations of the partitioned models for 6-Attack-Argus and 6-Attack-Tranalyzer can be seen in Figures 5.4 and 5.5 respectively. Their accuracies can be found in Tables 5.6 and 5.7 respectively.

The accuracy tables show that the models for both Argus and Tranalyzer predict

Table 5.4: Performance measures for WEKA decision tree classification of 10-fold cross-validated 6-Attack-Argus dataset.

	TP	FP Rate	Precision	Recall	F-Score	ROC Area	Class
	1	0	1	1	1	1	greip-argus
	1	0	1	1	1	1	syn-argus
	1	0	1	1	1	1	ack-argus
	1	0	1	1	1	1	greeth-argus
	1	0	1	1	1	1	vse-argus
	1	0	1	1	1	1	udp-argus
Weighted Avg.	1	0	1	1	1	1	

```
numHdrs <= 4
|   tcpMSS <= 0
|   |   tcpInitWinSz <= 229
|   |   |   minPktSz <= 25
|   |   |   |   flowInd <= 16
|   |   |   |   |   tcpTmS <= 1618685350
|   |   |   |   |   |   tcpTmS <= 385073315: vse-tran (16.0)
|   |   |   |   |   |   tcpTmS > 385073315: greip-tran (3.0/1.0)
|   |   |   |   |   tcpTmS > 1618685350
|   |   |   |   |   |   connSipDprt <= 4: udp-tran (3.0/1.0)
|   |   |   |   |   |   connSipDprt > 4: syn-tran (3.0/1.0)
|   |   |   |   flowInd > 16: vse-tran (63647.0)
|   |   |   minPktSz > 25: udp-tran (170405.0)
|   |   tcpInitWinSz > 229: ack-tran (193784.0)
|   tcpMSS > 0: syn-tran (193038.0)
numHdrs > 4
|   numHdrs <= 5: greip-tran (198392.0)
|   numHdrs > 5: greeth-tran (227524.0)
```

Figure 5.3: Decision tree generated by 10-fold cross validation on the 6-Attack dataset using Tranalyzer features.

Table 5.5: Performance measures for WEKA decision tree classification of 10-fold cross-validated 6-Attack-Tranalyzer dataset.

	TP	FP Rate	Precision	Recall	F-Score	ROC Area	Class
	1	0	1	1	1	1	greip-argus
	1	0	1	1	1	1	syn-argus
	1	0	1	1	1	1	ack-argus
	1	0	1	1	1	1	greeth-argus
	1	0	1	1	1	1	vse-argus
	1	0	1	1	1	1	udp-argus
Weighted Avg.	1	0	1	1	1	1	

```
sMeanPktSz <= 554
|   sMeanPktSz <= 80
|   |   Proto <= 6: syn-argus (193043.0/4.0)
|   |   Proto > 6: vse-argus (63662.0)
|   sMeanPktSz > 80: udp-argus (170405.0)
sMeanPktSz > 554
|   sMeanPktSz <= 578
|   |   Proto <= 6: ack-argus (193784.0)
|   |   Proto > 6: greip-argus (198392.0)
|   sMeanPktSz > 578: greeth-argus (227524.0)
```

Figure 5.4: Decision tree generated by running 66/33 (train/test) split on the 6-Attack dataset using Argus features.

Table 5.6: Performance measures for WEKA decision tree classification of the 6-Attack-Argus dataset.

	TP	FP Rate	Precision	Recall	F-Score	MCC	ROC Area	PRC Area	Class
	1	0	1	1	1	1	1	1	greeth-argus
	1	0	1	1	1	1	1	1	syn-argus
	1	0	1	1	1	1	1	1	udp-argus
	1	0	1	1	1	1	1	1	greip-argus
	1	0	1	1	1	1	1	1	vse-argus
	1	0	1	1	1	1	1	1	ack-argus
Weighted Avg.	1	0	1	1	1	1	1	1	

```
numHdrs <= 4
|   tcpMSS <= 0
|   |   tcpInitWinSz <= 229
|   |   |   minPktSz <= 25
|   |   |   |   flowInd <= 16
|   |   |   |   |   tcpTmS <= 1618685350
|   |   |   |   |   |   tcpTmS <= 385073315: vse-tran (16.0)
|   |   |   |   |   |   tcpTmS > 385073315: greip-tran (3.0/1.0)
|   |   |   |   |   tcpTmS > 1618685350
|   |   |   |   |   |   connSipDprt <= 4: udp-tran (3.0/1.0)
|   |   |   |   |   |   connSipDprt > 4: syn-tran (3.0/1.0)
|   |   |   |   flowInd > 16: vse-tran (63647.0)
|   |   |   minPktSz > 25: udp-tran (170405.0)
|   |   tcpInitWinSz > 229: ack-tran (193784.0)
|   tcpMSS > 0: syn-tran (193038.0)
numHdrs > 4
|   numHdrs <= 5: greip-tran (198392.0)
|   numHdrs > 5: greeth-tran (227524.0)
```

Figure 5.5: Decision tree generated by running 66/33 (train/test) split on the 6-Attack dataset using Tranalyzer features.

Table 5.7: Performance measures for WEKA decision tree classification of the 6-Attack-Tranalyzer dataset.

	TP	FP Rate	Precision	Recall	F-Score	MCC	ROC Area	PRC Area	Class
	1	0	1	1	1	1	1	1	greeth-argus
	1	0	1	1	1	1	1	1	syn-argus
	1	0	1	1	1	1	1	1	udp-argus
	1	0	1	1	1	1	1	1	greip-argus
	1	0	1	1	1	1	1	1	vse-argus
	1	0	1	1	1	1	1	1	ack-argus
Weighted Avg.	1	0	1	1	1	1	1	1	

with near perfect accuracy. Upon closer inspection, the confusion matrices show that only a few flows were classified incorrectly. The decision tree diagrams show that the trees of both flow exporters are very small. The Argus trees have 11 nodes, while the Tranalyzer trees are a bit bigger, with 19 nodes. Smaller trees are desirable if the prediction accuracy is still higher because they make for simpler human-understandable rules. They also make for more maintainable IDS rules.

5.5 Phase 2 - Decision Tree Classification of Mixed Datasets

Since the labelled CTU dataset is only available for Argus, only Argus was used in phase 2. However, phase 2 saw a more complicated data partitioning problem. Since the 'normal' class only had 900 data points, additional measures were needed to ensure the training was unbiased. This was a problem because the other classes had up to 250 times more samples. This led to two different methods of partitioning the data, one approach balanced, the other unbalanced.

The balanced approach involved taking 800 samples from each class and combining them to make the training set. The rest of the samples were used in testing. This insured that every class was equally represented in training. The drawback to this method is that it only leaves 100 test samples for the smallest class. The unbalanced approach simply mixed the CTU and 6-Attack datasets and did a 66/33 as in phase 1. The obvious drawback to this method is that the smaller classes might not be equally represented in the training and testing partitions. The trees of the balanced and unbalanced mixed datasets can be seen in Figures 5.6 and 5.7 respectively. Their accuracies can be found in Tables 5.8 and 5.9 respectively.

Both models performed almost perfectly on the mixed dataset. The balanced model achieved F-scores of 1 for all 6-Attack classes and scores of 0.981 and 0.997 for the Botnet and Normal classes respectively. The unbalanced model also achieved F-scores of 1 for all 6-Attack classes and scores of 0.995 and 0.999 for Botnet and Normal classes respectively. The similarity of the results between the balanced and unbalanced models shows that the training was successful in accurately representing the full dataset. The trees generated were also small, with the unbalanced and balanced trees having 19 and 15 nodes respectively. Although the results of the CART algorithm are not discussed here, it is worth noting that because CART only produces

```
sMinPktSz <= 136
|   sTtl <= 63: Normal (800.0)
|   sTtl > 63
|   |   sTtl <= 64
|   |   |   Proto <= 6: syn-argus (800.0)
|   |   |   Proto > 6: vse-argus (800.0)
|   |   sTtl > 64: Botnet (800.0)
sMinPktSz > 136
|   TotBytes <= 571
|   |   Proto <= 6: ack-argus (800.0)
|   |   Proto > 6: udp-argus (800.0)
|   TotBytes > 571
|   |   TotBytes <= 582: greip-argus (800.0)
|   |   TotBytes > 582: greeth-argus (800.0)
```

Figure 5.6: Decision tree generated from balanced mixed dataset.

Table 5.8: Performance measures for WEKA decision tree classification of the balanced mixed dataset.

	TP	FP Rate	Precision	Recall	F-Score	MCC	ROC Area	PRC Area	Class
	1	0	1	1	1	1	1	1	udp-argus
	1	0	1	1	1	1	1	1	ack-argus
	1	0	1	1	1	1	1	1	vse-argus
	1	0	1	1	1	1	1	1	greip-argus
	1	0	1	1	1	1	1	1	syn-argus
	1	0	0.962	1	0.981	0.981	1	0.962	Botnet
	0.995	0	1	0.995	0.997	0.997	0.997	0.995	Normal
	1	0	1	1	1	1	1	1	greeth-argus
Weighted Avg.	1	0	1	1	1	1	1	1	

```
sMinPktSz <= 554
|   sMinPktSz <= 535
|   |   sMaxPktSz <= 67
|   |   |   sTtl <= 64
|   |   |   |   DstPkts <= 0: vse-argus (63662.0)
|   |   |   |   DstPkts > 0: Normal (48.0)
|   |   |   sTtl > 64: Botnet (452.0/1.0)
|   |   sMaxPktSz > 67
|   |   |   DstPkts <= 0: syn-argus (193044.0/6.0)
|   |   |   DstPkts > 0
|   |   |   |   sTtl <= 126: Normal (4625.0/5.0)
|   |   |   |   sTtl > 126: Botnet (451.0/1.0)
|   sMinPktSz > 535: udp-argus (170405.0)
sMinPktSz > 554
|   sMeanPktSz <= 583.778931
|   |   Proto <= 6: ack-argus (193784.0)
|   |   Proto > 6: greip-argus (198392.0)
|   sMeanPktSz > 583.778931: greeth-argus (227524.0)
```

Figure 5.7: Decision tree generated from unbalanced mixed dataset.

Table 5.9: Performance measures for WEKA decision tree classification of the unbalanced mixed dataset.

	TP	FP Rate	Precision	Recall	F-Score	MCC	ROC Area	PRC Area	Class
	1	0	1	1	1	1	1	1	vse-argus
	1	0	1	1	1	1	1	1	syn-argus
	1	0	1	1	1	1	1	1	greeth-argus
	1	0	1	1	1	1	1	1	ack-argus
	1	0	1	1	1	1	1	1	greip-argus
	1	0	1	1	1	1	1	1	udp-argus
	0.998	0	1	0.998	0.999	0.999	0.999	0.998	Normal
	1	0	0.99	1	0.995	0.995	1	0.99	Botnet
Weighted Avg.	1	0	1	1	1	1	1	1	

binary trees, it generally creates larger trees.

The raw results and Python scripts used can be found on Github [17].

[17] Mirai Thesis Github repository: https://github.com/MackPreston/mirai-thesis

Chapter 6

Conclusion and Future Work

6.1 Conclusion

A virtual testbed network was setup in a lab environment, resulting in the successful generation and capture of Mirai DDoS attack traffic. The resulting traffic was tested against Snort IDS, a current intrusion detection solution. The results showed that current publicly available rulesets for Snort are inadequate for detecting Mirai attack traffic. Argus and Tranalyzer were used to export flows from the captured traffic, resulting in better features to analyze. Finally, the C4.5 and CART decision tree algorithms were used to accurately classify Mirai traffic by attack type and from normal and other botnet traffic.

6.2 Future Work

This thesis focused on Mirai DDoS attack traffic, but more work should be done on detecting Mirai bot scanning and infecting of new hosts. As discussed earlier, issues with Telnet in the virtual network prevented the spread of the malware in this research. Future works should avoid using a single bus interface and instead connect each device to a new interface on the router with a unique internal network. The issue could also have been caused by incompatibilities between Mirai's custom Telnet Implementation and Lubuntu's Telnet clients and servers. Fixing these issues could allow future works to examine scanner and loader behavior.

Furthermore, only Snort IDS was tested in these experiments. Future work could test traffic against different rulesets and different IDS platforms such as Bro and Suricata. The decision tree algorithm's solutions could also be hand-crafted for all Mirai attack types as an alternative to the work proposed in [9]. These rules could then be thoroughly tested against the collected Mirai traffic. There is also an opportunity to test the rules against the CTU datasets for false-positives. This was not accomplished

in this thesis because background traffic was filtered out of CTU flow dataset. Since Snort cannot process Argus and Tranalyzer flow files directly, the same filtering would somehow have to be performed on the raw packets in the pcap files. These packets could still be used to test the false-positive rate, but the results could not be directly compared to the results of this thesis.

One of the greatest difficulties with signature-based detection systems is keeping up with the massive number of variants of the malware that emerge. Researchers have already found numerous variants of Mirai active on the Internet, with attackers constantly changing and adding functionality [1]. Many of the rules generated in this thesis and in IDS rulesets could be deemed useless if an attacker modifies Mirai's source code. For this reason, it is important for future work to look at building robust rules that work across variants. Another solution would be to build systems that decrease the turnaround time on signature creation and testing [2]. Systems could be built that can create Snort rules directly from decision tree results.

It should be noted here that the analysis of the CTU-13 dataset was limited to only one of 13 scenarios. Future work could expand this analysis to cover the full CTU-13 dataset. It would also be useful to find a dataset that contains Mirai traffic (or at least suspected Mirai traffic) from an actual attack scenario. This way, Mirai traffic from the wild and from the lab could be compared, verifying that the suspected Mirai traffic matches the labs. This could also verify that Mirai attack traffic collected in this thesis was not influenced by the nature of the small virtual environment. It would also be useful to verify that Mirai traffic from IoT devices is identical to that generated by i386 virtual machines.

More work could also be done on feature selection. Numerous features were dropped in pre-processing due to non-numerical data types. Most of these, such as 'direction', have a very small set of possible values and could easily be encoded as categorical features. Due to time constraints, this research moved on when only six of the 10 Mirai attacks were successful. Future works could attempt to fix and capture the other four attacks. One more concern is that Snort does not seem to support rules containing some features calculated by Argus and Tranalyzer. These features could be removed from the feature sets used in training, or other IDS and mitigation strategies could be investigated.

Finally, the machine learning algorithms used in this thesis were very successful in classification. However, future work could include evaluating other machine learning algorithms such as Neural Networks or Genetic Algorithms. Improved validation methods such as stratified cross-validation could also be utilized.

Bibliography

[1] Manos Antonakakis, Tim April, Michael Bailey, Mathew Bernhard, Elie Bursztein, Jamie Cochran, Zakir Durumeric, J. Alex Halderman, Luca Invernizzi, Michalis Kallitsis, Deepak Kumar, Chaz Lever, Zane Ma, Joshua Mason, Damian Menscher, Chad Seaman, Nick Sullivan, Kurt Thomas, and Yi Zhou. Understanding the mirai botnet. USENIX Security Symposium, 2017.

[2] Naser Fallahi, Ashkan Sami, and Morteza Tajbakhsh. Automated flow-based rule generation for network intrusion detection systems. 24th Iranian Conference on Electrical Engineering (ICEE), 2016.

[3] Juan Guarnizo, Amit Tambe, Suman Sankar Bhunia, Martin Ochoa, Mils Ole Tippenhauer, Asaf Shabtai, and Yuval Elovici. Siphon: Towards scalable high-interaction physical honeypots. In *Proceedings of the 3rd ACM Workshop on Cyber-Physical System Security*. ACM, 2017.

[4] Fariba Haddadi and A. Nur Zincir-Heywood. Benchmarking the effect of flow exporters and protocol filters on botnet traffic classification. IEEE Systems Journal, 2016.

[5] Jiawei Han, Micheline Kamber, and Jian Pei. *Data Mining*. Morgan Kaufmann, 2011.

[6] James A. Jerkins. Motivating a market or regulatory solution to iot insecurity with the mirai botnet code. In *Computing and Communication Workshop and Conference (CCWC), 2017 IEEE 7th Annual*. IEEE, 2017.

[7] Richard Kirkby, Eibe Frank, and Peter Reutemann. *Weka explorer user guide for version 3-5-8*.

[8] Martin Roesch and Chris Green. *Snort User Manual 2.9.11*.

[9] Hamdija Sinanovic and Sasa Mrdovic. Analysis of mirai malicious software. `http://people.etf.unsa.ba/~smrdovic/publications/SoftCOM2017_Sinanovic_Mrdovic.pdf`, 2017. Online; accessed October 2017.

Appendix A

Confusion Matrices

Table A.1: WEKA 10-fold cross-validated decision tree confusion matrix for 6-Attack-Argus dataset.

		Classified as					
		greip	syn	ack	greeth	vse	udp
Actual Class	greip	198392	1	0	0	0	0
	syn	0	193038	0	0	1	0
	ack	0	1	193784	0	0	0
	greeth	0	0	0	227524	0	0
	vse	0	0	1	0	63662	0
	udp	0	0	1	0	0	170405

Table A.2: WEKA decision tree confusion matrix for 6-Attack-Argus dataset.

		Classified as					
		greeth	syn	udp	greip	vse	ack
Actual Class	greeth	77326	0	0	0	0	0
	syn	0	65260	0	0	0	1
	udp	0	0	58278	0	0	1
	greip	0	0	0	67664	0	0
	vse	0	0	0	0	21705	1
	ack	0	0	0	0	0	65679

Table A.3: WEKA 10-fold cross-validated decision tree confusion matrix for 6-Attack-Tranalyzer dataset.

		Classified as					
		udp	ack	greip	greeth	syn	vse
Actual Class	udp	170405	2	0	0	0	0
	ack	1	193784	0	0	1	0
	greip	0	0	198392	0	0	2
	greeth	0	0	0	227524	0	0
	syn	0	2	0	0	193038	0
	vse	0	0	2	0	0	63662

Table A.4: WEKA decision tree confusion matrix for 6-Attack-Tranalyzer dataset.

		Classified as					
		udp	ack	greip	greeth	syn	vse
Actual Class	udp	57891	0	0	0	2	0
	ack	0	66047	0	0	0	0
	greip	0	0	67673	0	0	1
	greeth	0	0	0	77349	0	0
	syn	0	0	0	0	65570	0
	vse	0	0	0	0	0	21384

Table A.5: WEKA 10-fold cross-validated decision tree confusion matrix for unbalanced mixed dataset.

		Classified as							
		syn	greip	ack	greeth	udp	vse	normal	botnet
Actual Class	syn	193038	0	0	0	0	0	1	0
	greip	0	198392	0	0	0	0	1	0
	ack	0	0	193782	2	0	0	1	0
	greeth	0	0	0	227524	0	0	0	0
	udp	0	0	0	0	170405	0	1	0
	vse	0	0	0	0	0	63662	1	0
	normal	3	0	0	0	2	0	4668	3
	botnet	0	0	0	0	0	0	0	901

Table A.6: WEKA decision tree confusion matrix for unbalanced mixed dataset.

		Classified as							
		vse	syn	greeth	ack	greip	udp	normal	botnet
Actual Class	vse	21585	0	0	0	0	0	0	0
	syn	0	65826	0	0	0	0	0	0
	greeth	0	0	77145	0	0	0	0	0
	ack	1	0	0	65999	0	0	0	0
	greip	1	0	0	0	67270	0	0	0
	udp	0	0	0	0	0	58047	0	0
	normal	0	0	0	0	0	0	1635	3
	botnet	0	0	0	0	0	0	0	300

Table A.7: WEKA decision tree confusion matrix for balanced mixed dataset.

		Classified as							
		udp	ack	vse	greip	syn	botnet	normal	greeth
Actual Class	udp	169599	0	0	0	1	0	0	6
	ack	0	192982	0	0	1	0	0	2
	vse	0	0	62862	0	1	0	0	0
	greip	0	0	0	197592	1	0	0	0
	syn	0	0	0	0	192239	0	0	0
	botnet	0	0	0	0	0	101	0	0
	normal	3	0	12	0	0	4	3855	2
	greeth	0	0	0	0	0	0	0	226724